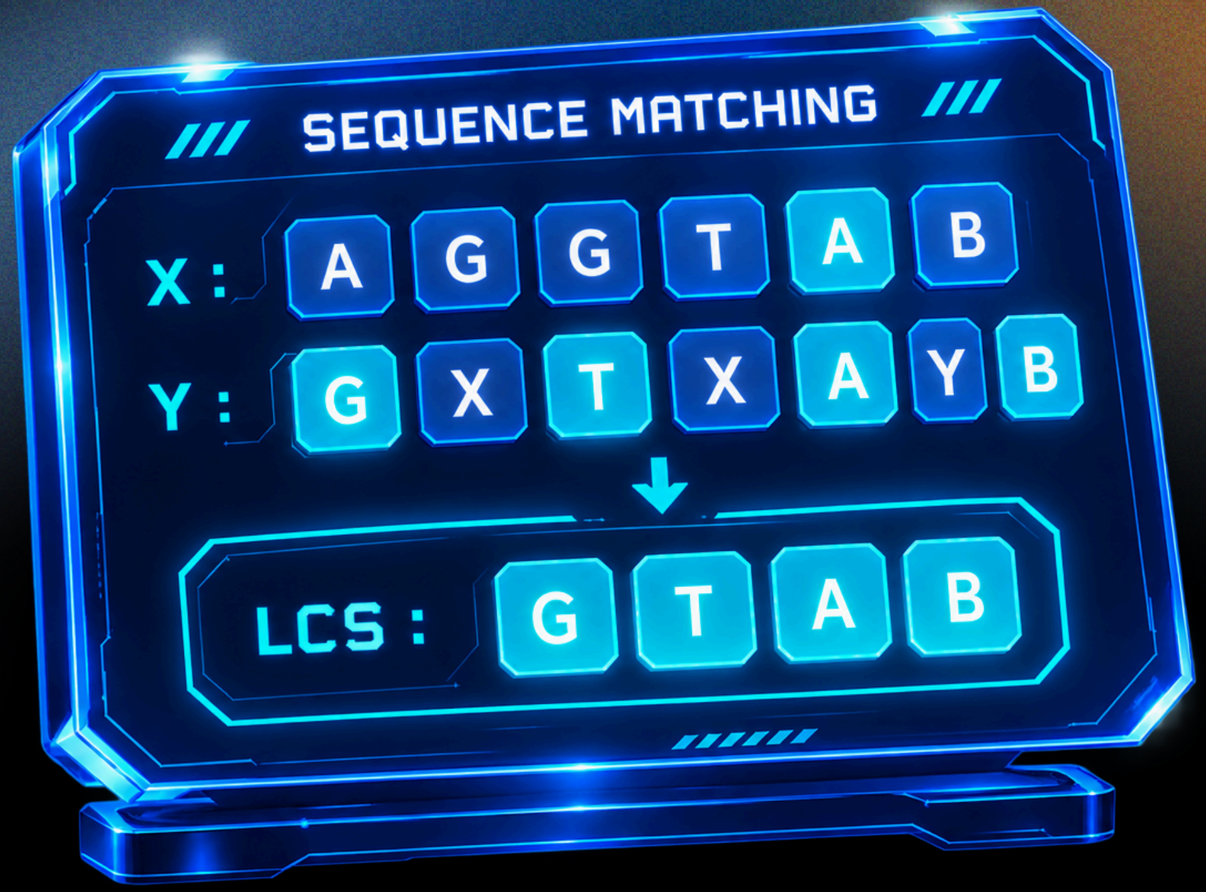
Longest Common Subsequence

Sanskruti Bramhankar
25070521260

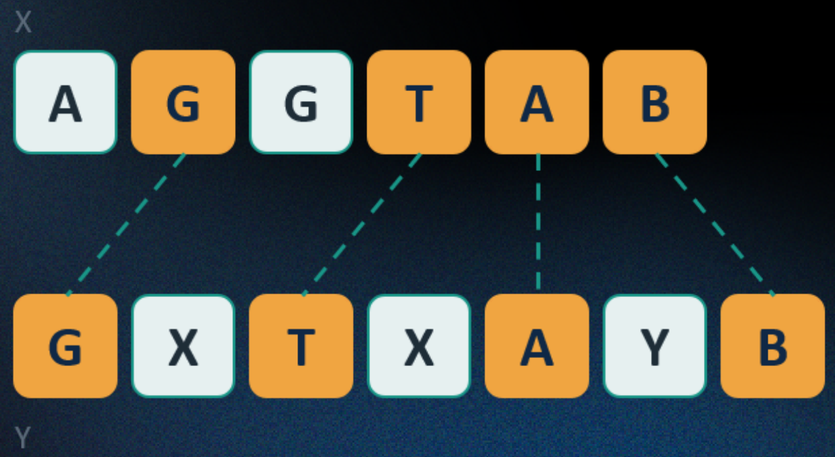
Made using : Canva
and chatgpt



INTRODUCTION

The Longest Common Subsequence (LCS) is a fundamental problem in computer science ued to compare two sequences and identify the longest sequence of elements that appears in both in the same relative order.

- A subsequence is formed by deleting zero or more elements from a sequence.
- The remaining elements must maintain their original order.
- The selected elements do not need to be adjacent.
- LCS can be solved efficiently using dynamic programming.
- Example:
  String X = ABCDEF
  String Y = AEBDF
  One LCS = ABF
  Length of LCS = 3
  Note: For these strings, AB



LCS = "GTAB"

Length 4. An LCS may not be unique; the length always is.

SUBSEQUENCE VS. SUBSTRING

SUBSEQUENCE

String: ABCDEF

Example: ACE

A, C and E appear in order, but not next to one another.

SUBSTRING

String: ABCDEF

Example: BCD

A substring must contain consecutive characters.

LCS ALLOWS GAPS; A SUBSTRING DOES NOT.

PROBLEM STATEMENT : FIND LCS

INPUT STRING X

A G G T A B

INPUT STRING Y

G X T X A Y B

COMPARE IN ORDER

LCS = G T A B | LENGTH = 4

WHY LCS MATTERS

TEXT COMPARISON

Identify common ordered content between two texts.

VERSION CONTROL

Help detect shared lines and changes between versions.

BIOINFORMATICS

Compare DNA or protein sequences.

CORE USE

LCS provides a foundation for comparing ordered sequences.

BRUTE FORCE: WHY IT BECOMES EXPENSIVE

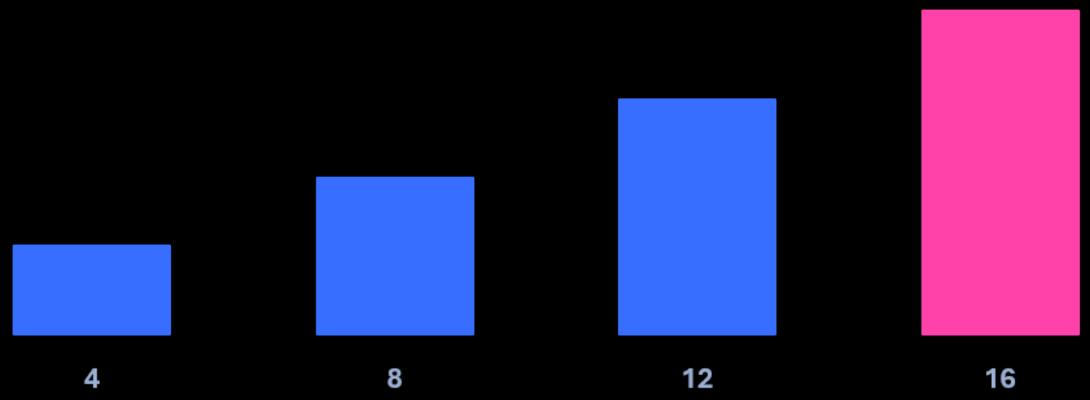
BRUTE-FORCE IDEA

Generate subsequences of one string and check which also appear in the other.

COST

A string of length n has 2^n possible subsequences.

Exponential growth



2^n

subsequences of a string of length n

DYNAMIC PROGRAMMING APPROACH

OVERLAPPING SUBPROBLEMS

The same prefixes are compared repeatedly.

OPTIMAL SUBSTRUCTURE

A best answer can be built from smaller prefix answers.

DP TABLE

Store answers for every prefix pair in C[i][j].

PREFIX PAIR → RECURRENCE → TABLE → LCS LENGTH

LCS RECURRENCE RELATION

BASE CASE

C[i][j] = 0 when i = 0 or j = 0

MATCH

If X[i−1] = Y[j−1], then C[i][j] = C[i−1][j−1] + 1

MISMATCH

Otherwise, C[i][j] = max(C[i−1][j], C[i][j−1])

EXAMPLE: BUILD THE ANSWER STEP BY STEP

01

Initialize

First row and first column are zero.

02

Compare

Check the current characters of X and Y.

03

Fill

Match: diagonal + 1.
Mismatch: max(top, left).

04

Read

The bottom-right cell gives the LCS length.

Final result for AGGTAB and GXTXAYB: GTAB

DYNAMIC PROGRAMMING TABLE

X\Y	ε	G	X	T	X	A	Y	B
ε	0	0	0	0	0	0	0	0
A	0	0	0	0	0	1	1	1
G	0	1	1	1	1	1	1	1
G	0	1	1	1	1	1	1	1
T	0	1	1	2	2	2	2	2
A	0	1	1	2	2	3	3	3
B	0	1	1	2	2	3	3	4

Bottom-right value = 4 → LCS length is 4

ALGORITHM: BOTTOM-UP DYNAMIC PROGRAMMING

```
LCS(X, Y):
  m = length(X), n = length(Y)
  create C[0..m][0..n]
  set first row and column to 0
  for i = 1 to m:
    for j = 1 to n:
      if X[i-1] == Y[j-1]:
        C[i][j] = C[i-1][j-1] + 1
      else:
        C[i][j] = max(C[i-1][j], C[i][j-1])
  return C[m][n]
```

RECOVERING THE ACTUAL LCS

START AT C[m][n]

Move backwards through the table, starting at the bottom-right cell.

WHEN CHARACTERS MATCH

Record the character and move diagonally up-left.

WHEN THEY DO NOT MATCH

Move toward the neighbouring cell with the larger value.

REVERSE RECORDED CHARACTERS

Traceback finds the sequence in reverse order.

TIME AND SPACE COMPLEXITY

TIME COMPLEXITY

$O(m \times n)$

Each cell is computed once.

SPACE COMPLEXITY

$O(m \times n)$

The full DP table is stored.

OPTIMIZATION

If only the LCS length is required, rolling rows can reduce auxiliary space to $O(\min(m, n))$.

REAL-WORLD APPLICATIONS

DIFF TOOLS

Find common ordered lines between files.

BIOINFORMATICS

Compare DNA or protein sequences.

DOCUMENT ANALYSIS

Measure common ordered content in text.

VERSION CONTROL

Support comparison of revisions and changes.

LCS

EFFICIENT SEQUENCE COMPARISON WITH DYNAMIC PROGRAMMING

REUSE

Store subproblem results.

OPTIMIZE

Avoid exponential search.

APPLY

Compare ordered sequences.

THANK YOU

Sanskruti Bramhankar • PRN 25070521260 • DAA